AUTONOMOUS

SRI MUTHUKUMARAN INSTITUTE OF TECHNOLOGY

APPROVED BY AICTE | AFFILIATED TO ANNA UNIVERSITY

CS25C01 - COMPUTER PROGRAMMING: C

LABORATORY RECORD

I YEAR / I SEMESTER

REGULATION 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SYLLABUS

CS25C01	Computer Programming: C	L	T	P	C
		2	0	2	3

Course Objectives:

- To equip engineering students with the foundational knowledge and practical skills in „C" programming to analyse and solve computational problems effectively.
- To foster problem-solving, critical thinking, and modular programming skills essential for engineering domains.

Introduction to C: Problem Solving, Problem Analysis Chart, Developing an Algorithm, Flowchart and Pseudocode, program structure, Compilation & Execution process, Interactive and Script mode, Comments, Indentation, Error messages, Primitive data types, Constants, Variables, Reserved words, Arithmetic, Relational, Logical, Bitwise, Assignment, Conditional operators, Input / Output Functions, Built-in Functions.

Practical: Create Problem Analysis Charts, Flowcharts and Pseudocode for simple C programs (Minimum three).

Control Structures: if, if-else, nested if, switch-case, while, do-while, for, nested loops, Jump statements.

Practical: Usage of conditional logics in programs. (Minimum three)

Functions: Function Declaration, Definition and Calling, Function Parameters and Return Types, Call by Value and Call by Reference, Recursive Functions, Scope and Lifetime of Variables, Header files and Modular Programming.

Practical: Usage of functions in programs. (Minimum three)

Strings & Pointers: One-dimensional and Multi-dimensional Arrays, Array operations and traversals, String Handling: String declaration, input/output, string library functions, Pointer arithmetic, Pointers and Arrays, Pointers to function, Dynamic memory allocation.

Practical: Programs using pointers, dynamic memory, pointer arithmetic, string manipulations, array operations. (Minimum three)

Structures & Unions: Defining and using structures, Array of structures, Pointers to structures, Unions and their uses, Enumerations.

Practical: Program to use structures and unions

File Operations: Open, read, write, close file operations, Binary vs Text files, File pointers, Error handling in file operations.

Practical: Programs reading/writing data in text and binary files (Minimum three).

Standard Libraries & Header Files: Using standard libraries like stdio.h, stdlib.h, string.h, math.h, Creating and using user-defined header files and libraries.

Practical: Use of standard and user-defined libraries in solving problems. (Minimum three), Project (Minimum Two)

Weightage: Continuous Assessment: 50%, End Semester Examinations: 50%

Assessment Methodology: Quiz (5%), Project (15%), Assignment Programs (25%), Practical (25%), Internal Examinations (30%)

References:

1. Thareja, R. (2021). Programming in C . Oxford University Press.
2. Balagurusamy, E. (2019). Programming in ANSI C. McGraw Hill Education.
3. Kanetkar, Y. (2020). Let us C. BPB Publications.
4. Kalicharan, N. (2022). Learn to program with C: An introduction to programming using the C language. Apress.
5. Forouzan, B. A., & Afyouni, H. (2023). Computer science: A structured programming approach in C (4th ed.). Cengage.

E-resources:

1. Learn-C.org - https://www.learn-c.org/
2. GeeksforGeeks - C Programming - https://www.geeksforgeeks.org/c-programming-language/
3. GNU C Library Documentation - https://www.gnu.org/software/libc/manual/
4. "Introduction to C Programming", Swayam MOOC Course, https://onlinecourses.swayam2.ac.in/imb25_mg71/

	Description of CO	**PO**	**PSO1**	**PSO2**	**PSO3**
CO1	Explain the potential usage of „C" in engineering applications	---			
CO2	To apply the concepts of „C" in solving engineering problems and formulate new projects.	PO1 (2) PO5 (2)			
CO3	To interpret the data and effectively communicate in groups.	PO2 (3) PO8 (1) PO9 (1)			
CO4	Adapt new programming concepts and technologies in the profession.	PO11 (1)			

INDEX

Ex.No	Date	Name of the Experiment	Page.No	Marks	Signature
1.a		**Program To Add Two Numbers**			
1.b		**Program To Largest Of Two Numbers**			
1.c		**Programs- Check Even Or Not**			
2.a		**Program To Demonstrate The Use Of If – Else Statement**			
2.b		**Program To Illustrate The Use Of If – Else If – Else Statement**			
2.c		**Program To Illustrate The Use Of Switch Case**			
2.d		**Find The Sum Of Individual Digits Of A Positive Integer**			
2.e		**Prime Number**			
3.a		**Fibonacci Series**			
3.b		**Armstrong Number Or Not**			
3.c		**Palindrome Of A Given Using While Loop**			
4.a		**C Program To Illustrate Function Call By Value**			
4.b		**C Program To Illustrate Call By Value Reference**			
4.c		**C Program To Pass Array To Functions**			
4.d		**To Find The Factorial Of A Given Integer By Using Recursive And Non-Recursive Functions**			
4.e		**To Find Gcd Of Two Given Integers By Using The Recurvise Function**			
4.f		**Find The Largest Number In A List Of Integers**			
4.g		**Program To Sort The Array In An Ascending Order**			
4.h		**C Program To Perform Addition Of Two Matrices**			
5.a		**Insert A String Into Another String From A Specified Position**			
5.b		**Program To Delete N Characters From A String**			
5.c		**C Program To Check Whether A String Is Palindrome Or Not**			
5.d		**C Program Counts Lines, Words, And Characters From A Given Text**			
6.a		**C Program To Illustrate Pointers To Structures**			
6.b		**Pointer Addition**			
6.c		**Pointer Subtraction**			
7		**Display Array Elements Using Calloc() Function**			

8.a		**Internal Marks Of Students**			
8.b		**Program to use structures and unions**			
9.a		**C Program To Display The Concept Of A File**			
9.b		**C Program To Copy The Contents Of One File To Another**			
9.c		**C Program To Reverse The First N Characters In A File**			
9.d		**Merge Two Files Into A Third File**			
10.a		**Use Of Standard And User-Defined Libraries In Solving Problems-Square Root (Standard) & Factorial (USER DEFINED)**			
10.b		**Use Of Standard And User-Defined Libraries In Solving Problems- Power Function (Standard)**			
10.c		**Use Of Standard And User-Defined Libraries In Solving Problems- String Length (Standard)**			
11.a		**Mini Project Create A —Railway Reservation System**			
11.b		**Banking Application**			

Ex.No:1.a	CREATE PROBLEM ANALYSIS CHARTS, FLOWCHARTS AND PSEUDOCODE FOR SIMPLE C PROGRAMS- PROGRAM TO ADD TWO NUMBERS
Date:	

AIM

Create problem analysis charts, flowcharts and pseudocode for add two numbers using c program.

PROBLEM ANALYSIS CHART (INPUT-PROCESS-OUTPUT - IPO CHART):

```
| **Input**          | **Process**       | **Output**  |
| -------------------| -------------     | -------- |
| Two numbers a, b   | sum = a + b       | Display sum |
```

ALGORITHM

Step 1: Start.

Step 2: Get the first number (num1).

Step 3: Get the second number (num2).

Step 4: Calculate the sum: sum = num1 + num2.

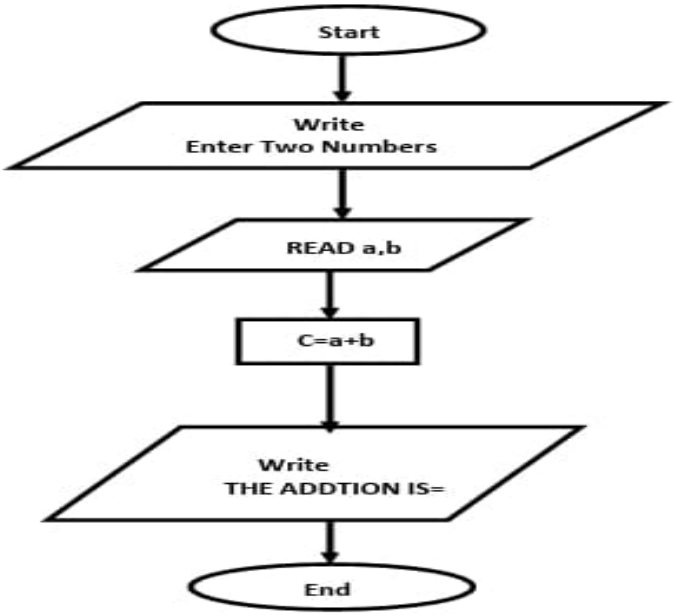
Step 5: Display the sum.

Step 6: Stop.

PSEUDOCODE

```
BEGIN
  READ num1
  READ num2
  CALCULATE sum = num1 + num2
  PRINT sum
END
```

FLOW CHART



PROGRAM

```
#include <stdio.h>
 void main()
{
int a,b,c;
printf("Enter two Numbers:");
scanf("%d%d",&a,&b);
 c=a+b;
printf("THE ADDTION IS=%d",c);
 }
```

Output:

Enter two Numbers: 100 90 THE ADDTION

IS=190

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:1.b	CREATE PROBLEM ANALYSIS CHARTS, FLOWCHARTS AND PSEUDOCODE FOR SIMPLE C PROGRAMS- PROGRAM TO LARGEST OF TWO NUMBERS
Date:	

AIM

Create problem analysis charts, flowcharts and pseudocode for Largest of Two Numbers using c program.

PROBLEM ANALYSIS CHART FOR LARGEST OF TWO NUMBERS

Component	Description
Input	Two distinct numbers (e.g., num1, num2).
Process	Compare num1 and num2 to determine which is greater.
Output	The larger of the two input numbers.
Constraints	Input numbers are numerical.

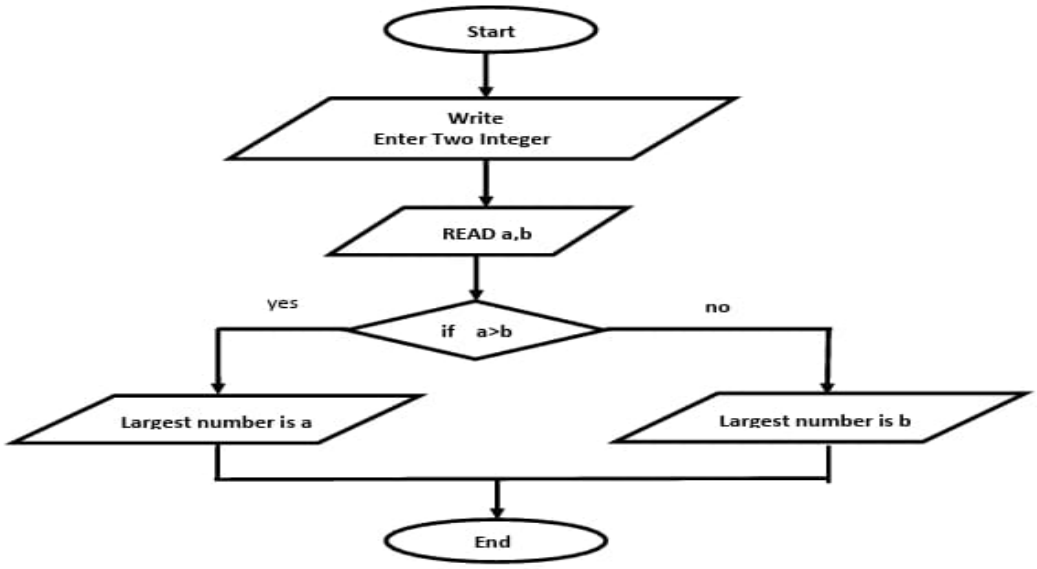
ALGORITHM

Step 1: START
Step 2: Read the first number, 'num1'.
Step 3: Read the second number, 'num2'.
Step 4: IF 'num1' is greater than 'num2' THEN
Step 5: Display 'num1' as the largest.
Step 6: ELSE
Step 7: Display 'num2' as the largest.
Step 8: END IF
Step 9: STOP

PSEUDOCODE

```
BEGIN
   DECLARE num1, num2 AS INTEGER
   DISPLAY "Enter the first number: "
   INPUT num1
   DISPLAY "Enter the second number: "
   INPUT num2
   IF num1 > num2 THEN
      DISPLAY num1 + " is the largest number."
   ELSE
      DISPLAY num2 + " is the largest number."
   END IF
END
```

FLOWCHART



PROGRAM

```
#include <stdio.h>

void main()
{
int a, b;
printf("Enter two integers :");
scanf("%d%d", &a, &b);
if (a > b)
{
printf("Largest number is %d.", a);
}
else
{
printf("Largest number is %d.", b);
}
}
```

OUTPUT:

Enter two integers :25 19

Largest number is 25

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:1.C	CREATE PROBLEM ANALYSIS CHARTS, FLOWCHARTS AND PSEUDOCODE FOR SIMPLE C PROGRAMS- CHECK EVEN OR NOT
Date:	

AIM

Create problem analysis charts, flowcharts and pseudocode for check even or not.

PROBLEM ANALYSIS CHART FOR LARGEST OF TWO NUMBERS

Input	**Process**	**Output**
One number n	If n % 2 == 0 → Even else Odd	Display result

ALGORITHM

Step 1: Start

Step 2: Get: an integer number from the user.

Step 3: Calculate: the remainder when number is divided by 2.

Step 4: If: the remainder is 0, then display "Even".

Step 5: Else: (if the remainder is not 0), then display "Odd".

Step 6: End

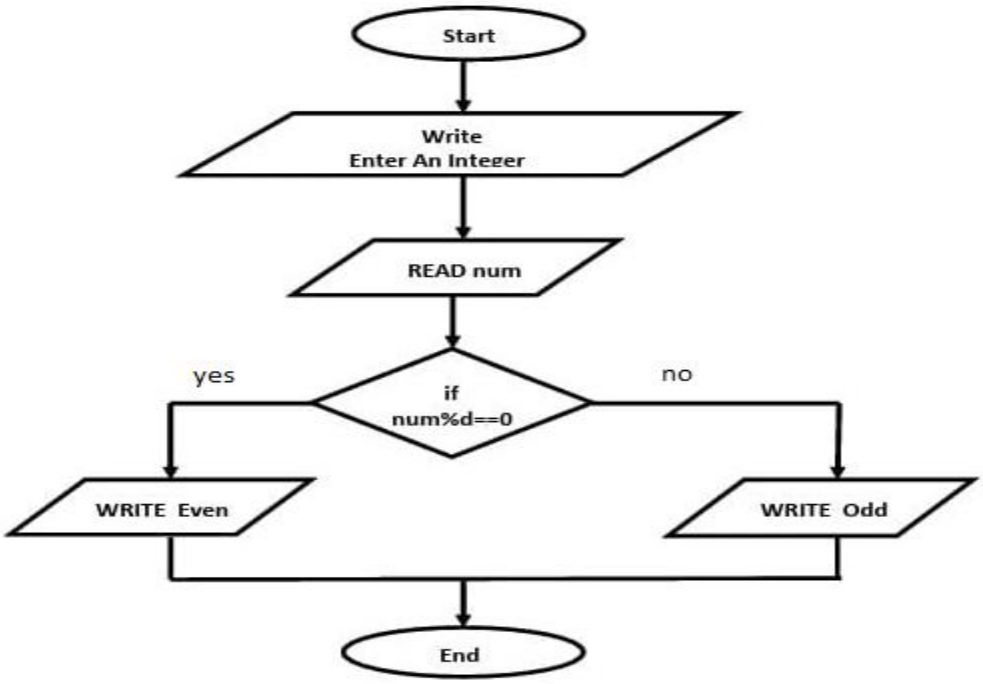
PSEUDOCODE

```
BEGIN
   DECLARE INTEGER number

   OUTPUT "Enter an integer: "
   INPUT number

   IF (number MOD 2 == 0) THEN
      OUTPUT "Even"
   ELSE
      OUTPUT "Odd"
   END IF
END
```

FLOWCHART



PROGRAM

```
#include <stdio.h>
void main()
 {
 int number;
 printf("Enter a number: ");
 scanf("%d", &number);
if (number % 2 == 0)
printf("%d is even.", number);
else
 printf("%d is odd.", number);
 }
```

OUTPUT

```
Enter an integer: 33
33 is odd.
```

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:2.a	USAGE OF CONDITIONAL LOGICS IN PROGRAMS- USE OF IF – ELSE STATEMENT
Date:	

AIM

To write a c program to demonstrate the use of if – else statement.

ALGORITHM

Step1: Start
Step 2: Declare and intialize the variables with user inputs
Step 3: Check the condition, if the condition is true the Corresponding statement is executed.
Step4: If the condition is false else part will get executed.
Step5: stop

PROGRAM

```
#include <stdio.h>
Int  main()
{
int number = 0;
printf("Enter a number: ");
scanf("%d", &number);
if (number % 2 == 0)
{
printf("%d is an even number\n", number);
}
else
{
printf("%d is an odd number\n", number);
}
return 0;
}
```

OUTPUT:

INPUT : Enter a number:4

OUTPUT : 4 is even number

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:2 b	USAGE OF CONDITIONAL LOGICS IN PROGRAMS- USE OF IF – ELSE IF – ELSE STATEMENT
Date:	

AIM

To write a c program to demonstrate the use of if – else if – else statement.

ALGORITHM

Step1: Start

Step 2: Declare and intialize the variables with user inputs

Step 3: Check the condition , if the condition is true the Corresponding statement is executed.

Step4: If the condition is false else if part will get executed., if nothing is true else part will excute.

Step5: stop

PROGRAM

```
#include <stdio.h>

int main()

{

int marks;
printf("Enter your marks: ");

 scanf("%d", &marks);

if (marks > 85 && marks <= 100)

{

printf("Congrats! You scored grade A...\n");

}
else if (marks > 60 && marks <= 85)

{
printf("You scored grade B+...\n");

}
else if (marks > 40 && marks <= 60)

{

printf("You scored grade B...\n");

}
else if (marks > 30 && marks <= 40)
{


printf("You scored grade C...\n");
```

```
}
else
{
printf("Sorry, you have failed...\n");
}
return 0;
}
```

OUTPUT:

INPUT : Enter your marks?10

OUTPUT : Sorry you are fail…

INPUT : Enter your marks?40

OUTPUT : you scored grade C…

INPUT : Enter your marks?90

OUTPUT : Congrats ! you scored grade A…

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:2 c	USAGE OF CONDITIONAL LOGICS IN PROGRAMS- PROGRAM TO ILLUSTRATE THE USE OF SWITCH CASE
Date:	

AIM

To write a c program to demonstrate the use of switch case

ALGORITHM

Step1: Start

Step 2: Declare and intialize the variables with user inputs

Step 3: Check the condition for switch case , if true execute the switch part

Step4: if it is false default part will be executed

Step5: stop

PROGRAM

```
#include <stdio.h>
 int main()
{
int number = 0;
printf("Enter a number: ");
scanf("%d", &number);
switch (number)
{
case 10:
printf("Number is equal to 10\n");
break;
case 50:
printf("Number is equal to 50\n");
break;
case 100:
printf("Number is equal to 100\n");
 break;
default:
printf("Number is not equal to 10, 50 or 100\n");
}
return 0;
}
```

OUTPUT:

INPUT : Enter a number :4

OUTPUT : number is not equals to 10,50 or 100

INPUT :Enter a number : 50

OUTPUT : number is equals to 50

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:2 d	USAGE OF CONDITIONAL LOGICS IN PROGRAMS- FIND THE SUM OF INDIVIDUAL DIGITS OF A POSITIVE INTEGER USING WHILE LOOP
Date:	

AIM

To write a C program to find the sum of individual digits of a positive integer using while loop.

ALGORITHM

STEP1: start

Step 2: Declare and intialize the variables n , sum=0

Step 3: scan the input of n from the user

Step4: use while loop to iterate and evaluate the sum of digits of integers

Step5: stop

PROGRAM

```
#include <stdio.h>

 int main()

{

int n, sum = 0;
printf("Enter a positive integer: ");

 scanf("%d", &n);

while (n > 0)

{

// checks the condition
sum = sum + n % 10;  // add last digit

n = n / 10;        // remove last digit

}
printf("Sum of individual digits = %d\n", sum);

 return 0;

}
```

OUTPUT:

Input : Enter any integer : 1234

output: sum of individual digits is : 10

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:2 e	USAGE OF CONDITIONAL LOGICS IN PROGRAMS- PRIME NUMBERS BETWEEN 1 AND N IS A VALUE SUPPLIED USING FOR LOOP
Date:	

AIM

To write a C program to generate all the prime numbers between 1 and n is a value supplied by the user

ALGORITHM

STEP1: start

Step 2: Declare and intialize the variables num ,i,count ,n

Step 3: scan the input of n from the user

Step4: usefor loop to iterate and generate all the prime numbers between 1 and n

Step5: stop

PROGRAM

```
#include <stdio.h>

 int main()
{
int num, i, count, n;
printf("Enter max range: ");
scanf("%d", &n);
for (num = 1; num <= n; num++)
{
count = 0;
for (i = 2; i <= num / 2; i++)
{
if (num % i == 0)
{
count++;

break;
}
}
if (count == 0 && num != 1)
```

```
{
printf("%d ", num);
}
}
return 0;
}
```

OUTPUT:

input : enter max range:15

output : 2 3 5 7 11 13

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:3 a	FIBONACCI SERIES
Date:	

AIM

To write a C program to generate fibonacci series.

ALGORITHM

STEP1: start
Step 2: Declare and intialize the variables f0, f1, f, n,i
Step 3: scan the input of n from the user
Step4: use while loop to iterate and generate the fibonacci series
step5:stop

PROGRAM

```
#include <stdio.h>

int main()

{

int f0, f1, f, n, i;
printf("ENTER THE VALUE FOR n: ");
scanf("%d", &n);

f0 = 0;

f1 = 1;

printf("FIBONACCI SEQUENCE FOR THE FIRST %d TERMS:\n", n);

i = 0;

while (i < n)

{
printf("%d\t", f0);

f = f0 + f1;

f0 = f1;

 f1 = f;

i = i + 1;

}
printf("\n");

 return 0;

}
```

OUTPUT :

input : enter the value for n=10

output : Fibonacci sequence for the first 10 terms:

0 1 1 2 3 5 8 13 21 34

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:3 b	ARMSTRONG NUMBER OR NOT
Date:	

AIM

To write a C program to check whether ARMSTRONG NUMBER OR NOT

ALGORITHM

STEP1: start

Step 2: Declare and intialize the variables n, n1, rem, num=0

Step 3: scan the input of n from the user

Step4: use while loop to iterate and check whether armstrong number or not

step5:stop

PROGRAM

```
#include <stdio.h>
int main()
{
int n, n1, rem, num = 0;
printf("Enter a positive integer: ");
scanf("%d", &n);
n1 = n;
while (n1 != 0)
{
rem = n1 % 10;
num += rem * rem * rem;
n1 /= 10;
}
if (num == n)
printf("%d is an Armstrong number.\n", n);
else
printf("%d is not an Armstrong number.\n", n);
return 0;
}
```

OUTPUT :

input : enter a positive integer=371

output : 371 is an Armstrong number

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:3 c	PALINDROME OF A GIVEN NUMBER USING WHILE LOOP
Date:	

AIM

To write a C program to check whether palindrome or not.

ALGORITHM

STEP1: start

Step 2: Declare and intialize the variables n,r,sum, temp=0

Step 3: scan the input of n from the user

Step4: use while loop to iterate and check whether palindrome or not

step5:stop

PROGRAM

```
#include <stdio.h>

int main()
{
int n, r, sum = 0, temp;
printf("Enter the number: ");
scanf("%d", &n);
temp = n;
while (n > 0)
{
r = n % 10;
sum = (sum * 10) + r;

n = n / 10;
}
if (temp == sum)
printf("%d is a palindrome number\n", temp);
 else
printf("%d is not a palindrome number\n", temp);
return 0;
}
```

OUTPUT:

input : enter the number=151

output : palindrome number

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 4 a	FUNCTIONS- TO ILLUSTRATE FUNCTION CALL BY VALUE
Date:	

AIM

To write a c program to pass parameters using call by value function.

ALGORITHM

Step 1: start

Step 2: declare and initialize a and b values

Step 3: define a function swapx() with two arguments of type val

Step 4: in the main function, call the swapx() with the value of a and b

Step 5: display the output of call by value

Step 6: stop

PROGRAM

```
#include <stdio.h>
// Function Prototype
void swapx(int x, int y);
int main()
{
int a = 10, b = 20;
// Pass by Value
swapx(a, b);
printf("In the Caller:\na = %d b = %d\n", a, b);
return 0;
}
// Swap function (Call by Value)
void swapx(int x, int y)
{
int t;
t = x;
x = y;
y = t;
printf("Inside swapx (Call by Value):\nx = %d y = %d\n", x, y);
}
```

OUTPUT:

Inside function: x=20 y=10

In the caller : a=10 b=20

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:4 b	FUNCTIONS-ILLUSTRATE CALL BY REFERENCE
Date:	

AIM

To write a c program to illustrate the use of call by reference.

ALGORITHM

Step 1: start

Step 2: declare and initialize a and b values

Step 3: define a function swapx() with two arguments of pointer type

Step 4: in the main function, call the swapx() with the address of a and b

Step 5: display the output of call by reference

Step 6: stop

PROGRAM

```
#include <stdio.h>

// Function Prototype
void swapx(int *x, int *y);

int main()

{
int a = 10, b = 20;

// Pass by reference

swapx(&a, &b);

printf("Inside the Caller:\na = %d b = %d\n", a, b);

return 0;

}
// Function to swap two variables by reference

 void swapx(int *x, int *y)

{

int t;
t = *x;

*x = *y;

*y = t;

printf("Inside the Function:\nx = %d y = %d\n", *x, *y);

}
```

OUTPUT:

Inside function: x=20 y=10

In the caller : a=20 b=10

RESULT

The C program was executed successfully, and its output was verified.

Ex.No:4 c	FUNCTIONS-PASS ARRAY TO FUNCTIONS
Date:	

AIM

To write a c program to illustrate to pass array to functions.

ALGORITHM

Step 1: start

Step 2: declare and define function named display that takes two integer arguments age1and age2

Step 3: In the main function, declare an integer array ageArray and initialize it with the values {2, 8, 4, 12}.

Step 4: Call the display function, passing the first and second elements of ageArray as arguments.

Step 5: End the main function by returning 0 to indicate successful execution.

PROGRAM

```
#include <stdio.h>
void display(int age1, int age2)
{
printf("%d\n", age1);
printf("%d\n", age2);
}
int main()
{
int ageArray[] = {2, 8, 4, 12};
// Pass second and third elements to display()
display(ageArray[1], ageArray[2]);
return 0;
}
```

OUTPUT:

2

8

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 4 d	FUNCTIONS-FACTORIAL USING RECURSIVE FUNCTION
Date:	

AIM

To write a C program to find the factorial of a given integer by using recursive and non-recursive functions.

RECURSIVE ALGORITHM

Step 1: Define the recursive function

Step 2: Read the number n

Step 3: if n is equal to 0

Step 4: then print “factorial of 0 is 1”

Step 5: else call the recursive function

Step 6: print the factorial value.

PROGRAM

```
#include <stdio.h>
// Recursive function prototype unsigned
int factorial(int n);
int main()
{
int n;
printf("Enter the number: ");
 scanf("%d", &n);
if (n < 0)
{
printf("Factorial is not defined for negative numbers.\n");

return 1;
}
if (n == 0)
printf("Factorial of 0 is 1\n");
 else
printf("Factorial of %d using recursive function is %u\n", n, factorial(n));
return 0;
}
```

```
// Recursive factorial function unsigned
int factorial(int n)
{
return (n >= 1) ? n * factorial(n - 1) : 1;
}
```

OUTPUT:

input: enter number: 5

output: factorial of 5 using recursive function is :120

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 4 e	FUNCTIONS- C PROGRAM TO FIND THE GCD OF TWO GIVEN INTEGERS BY USING THE RECURSIVE FUNCTION
Date:	

AIM:

To find the Gcd of two given integers by using the recursive function.

ALGORITHM (MAIN PROGRAM)

Step 1: start

Step 2: read a,b

Step 3: call the sub program GCD(a,b) for print the value

Step 4: stop

SUB PROGRAM: GCD(n,m)

Step 1: if n>m return GCD(n,m)

Step 2: if n==0 return m else goto step 3 Step 3:

return GCD (n,m%n)

Step 4: return to main program

PROGRAM

```
#include <stdio.h>
// Recursive function to find GCD
int gcdrecursive(int m, int n)
{
if (n > m)
return gcdrecursive(n, m);
if (n == 0)
return m;
 else
return gcdrecursive(n, m % n);
}
int main()
{
int a, b;
printf("Enter the two numbers whose GCD is to be found: ");
scanf("%d %d", &a, &b);
printf("GCD of %d and %d is %d\n", a, b, gcdrecursive(a, b));
return 0;
}
```

OUTPUT :

input : enter the two numbers whose gcd is to be found: 5 25

output : gcd of a,b is : 5

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 4 f	ARRAYS-FIND THE LARGEST NUMBER IN A LIST OF INTEGERS/
Date:	

AIM

To find the largest number in a list of integers.

ALGORITHM

Step 1: start

Step 2: read n

Step 3: initialize i=0

Step 4: if i<n do as follows. If not goto step 5 Read a[i]

Increment i Goto step 4

Step 5: small=a[0], large=a[0]

PROGRAM

```
#include <stdio.h>
int main()
{
int n;
double arr[100], largest;
printf("Enter the number of elements (1 to 100): ");
scanf("%d", &n);
if (n < 1 || n > 100)
{
printf("Invalid number of elements.\n");
return 1;
}
printf("Enter %d elements separated by space: ", n);
for (int i = 0; i < n; ++i)
{
scanf("%lf", &arr[i]);
}
largest = arr[0];
for (int i = 1; i < n; ++i)
{
```

```
if (arr[i] > largest)
{
largest = arr[i];
}
}
printf("Largest element = %.2lf\n", largest);
 return 0;
}
```

OUTPUT:

Input

```
enter the array size:10
enter the elements of array

7 10 9 8 7 6 5 4 3 2 1
```

output

```
the largest value is :10
```

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 4 g	ARRAYS - C PROGRAM TO SORT THE ARRAY IN AN ASCENDING ORDER
Date:	

AIM

To sort the array in ascending order.

ALGORITHM

Step 1: start

Step 2: declare and initialize i,a,j,n,number[30]

Step 3: read the value of n

Step 4: scan the array elements from the user

Step 5: with for loop iterate the array

Step 6: if the first element is lesser than the next element,pass

Step 7: else move it right to the element

Step 8: display the array

Step 9: stop

PROGRAM

```
#include <stdio.h>
 int main()
{
int i, j, a, n, number[30];
printf("Enter the value of N: ");
scanf("%d", &n);
printf("Enter %d numbers:\n", n);
for (i = 0; i < n; ++i)
{
scanf("%d", &number[i]);
}
// Sorting using simple bubble-sort logic
for (i = 0; i < n; ++i)
{
for (j = i + 1; j < n; ++j)
{
if (number[i] > number[j])
```

```
 {
 a = number[i];
 number[i] = number[j];
 number[j] = a;
 }
 }
 }
printf("The numbers arranged in ascending order are:\n");
for (i = 0; i < n; ++i)
 {
  printf("%d ", number[i]);
  }
  printf("\n");
  return 0;
  }
```

OUTPUT:

input : enter the value of N 5
enter the number
78

90

456

780

200

output : the numbers arranged in ascending order are given below

78 90 200 456 780

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 4 h	ARRAYS - TO PERFORM ADDITION OF TWO MATRICES
Date:	

AIM

To write a C program to perform addition of two matrices.

ALGORITHM

Step 1: Start

Step2: for i is 0 to 2 by step 1 for j is 0 to 2 by step 1

Step 3: Read a[i][j],b[i][j]

Step 4: goto step 2

Step 5: calculate c[i][j]=a[i][j]+b[i][j]

Step 6: goto step 2

Step 7: Print c[i][j]

Step 8: Stop

PROGRAM

```
#include <stdio.h>

 int main()
{
int a[3][3], b[3][3], c[3][3];
int i, j;
printf("Enter A matrix (3x3):\n");

for (i = 0; i < 3; i++)
{
for (j = 0; j < 3; j++)
{
scanf("%d", &a[i][j]);
}
}
printf("Enter B matrix (3x3):\n");
for (i = 0; i < 3; i++)
{
for (j = 0; j < 3; j++)
{
```

```
scanf("%d", &b[i][j]);
}
}
// Matrix Addition
 for (i = 0; i < 3; i++)
{
for (j = 0; j < 3; j++)
{
c[i][j] = a[i][j] + b[i][j];
}
}
printf("After addition of two matrices:\n");
for (i = 0; i < 3; i++)
{
for (j = 0; j < 3; j++)
{
printf("%d\t", c[i][j]);
}
printf("\n");
}
return 0;
}
```

OUTPUT:

input

enter A matrixenter B matrix 1 2 3 1 1 1

4 5 6 1 1 1

7 8 9 1 1 1

output

after addition of two matrices is.. 2 3 4

5 6 7

8 9 10

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 5 a	STRING MANIPULATION- INSERT A STRING INTO ANOTHER STRING FROM A SPECIFIED POSITION.
Date:	

AIM

To insert a string into another string from a specified position.

ALGORITHM

Step 1: start

Step 2: read main string and substring

Step 3: find the length of main string(r)

Step 4: find length of sub string(n)

Step 5: copy main string into sub string

Step 6: read the position to insert the sub string(p)

Step 7: copy sub string into main string from position p-1

Step 8: copy temporary string into main string from position p+n-1

Step 9: print the strings

Step 10: stop

PROGRAM

```
#include <stdio.h>
#include <string.h>
int main()
 {
char a[100], b[50], c[150];
int p, i, j, k;
printf("Enter first string: ");
 fgets(a, sizeof(a), stdin);
a[strcspn(a, "\n")] = '\0';
 // remove newline
 printf("Enter second string: ");
fgets(b, sizeof(b), stdin);
b[strcspn(b, "\n")] = '\0';
// remove newline
printf("Enter the position where the string has to be inserted: ");
scanf("%d", &p);
int lenA = strlen(a);
```

```
 int lenB = strlen(b);
if (p < 0 || p > lenA)
{
printf("Invalid position!\n"); return 1;
}
// Copy first part of a into c
for (i = 0; i < p; i++)
{
c[i] = a[i];
}
// Insert b into c
for (j = 0; j < lenB; j++)
{
c[i + j] = b[j];
}
// Copy rest of a into c
for (k = p; k <= lenA; k++)
{
c[lenB + k] = a[k];
}
printf("Result: %s\n", c);
return 0;
}
```

OUTPUT:

input

enter first string:ad enter second string:bc

enter the position where the item has to be inserted:1 output:

abcd

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 5 b	STRING MANIPULATION- PROGRAM TO DELETE N CHARACTERS FROM A STRING
Date:	

AIM

To delete n Characters from a given position in a given string.

ALGORITHM

Step 1: start

Step 2: read string

Step 3: find the length of the string

Step 4: read the value of number of characters to be deleted and positioned

Step 5: string copy part of string from position to end, and (position + number of characters to end)

Step 6: stop

PROGRAM

```
#include <stdio.h>
#include <string.h>
 int main()
{
char a[100], c[100];
int pos, n, i, j, L, La;
printf("Enter a string: ");
fgets(a, sizeof(a), stdin);
a[strcspn(a, "\n")] = '\0';
// remove newline
printf("Enter position for deletion (starting from 0): ");
scanf("%d", &pos);
printf("Enter number of characters to be deleted: ");
scanf("%d", &n);
La = strlen(a);
if (pos < 0 || pos >= La)
{
printf("Invalid position!\n");
return 1;
}
```

```
if (pos + n > La)
{
n = La - pos; // adjust deletion length if it goes out of bounds
}
L = pos + n;
// Copy part before deletion
for (i = 0; i < pos; i++)
{
c[i] = a[i];
}
// Copy part after deletion
j = pos;
for (i = L; i <= La; i++)
{
c[j] = a[i]; j++;
}
printf("String after Deletion is: %s\n", c);
return 0;
}
```

OUTPUT:

input:

enter the string : jayapal

enter the position from where to delete: 4

enter the number of characters to be deleted : 2 output:

jayal

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 5 c	STRING MANIPULATION- C PROGRAM TO CHECK WHETHER A STRING IS PALINDROME OR NOT
Date:	

AIM

To determine if the given string is palindrome or not.

DESCRIPTION

Palindrome means string on reversal should be same as original

Ex: madam on reversal is also madam

ALGORITHM

Step 1: start

Step 2: read string A

Step 3: copy string A into B

Step 4: reverse string B

Step 5: compare A &B If A equals B to goto step 6 Else goto step 7

Step 6: print given string A is palindrome

Step 7: print given string is not palindrome

PROGRAM

```
#include <stdio.h>
#include <string.h>
int main()
{
char string[100], reverse_string[100];

int i, length = 0, flag = 1;

printf("Enter a string: ");

fgets(string, sizeof(string), stdin);
string[strcspn(string, "\n")] = '\0';  // remove newline from fgets
// Calculate length

length = strlen(string);
printf("The length of the string '%s' = %d\n", string, length);
```

```
// Reverse the string
for (i = 0; i < length; i++)

{
reverse_string[i] = string[length - i - 1];

}

reverse_string[length] = '\0';
// Check for palindrome

for (i = 0; i < length; i++)

{
if (reverse_string[i] != string[i])

{
flag = 0;

break;

}

}

if (flag == 1)
printf("%s is a palindrome.\n", string);

else

printf("%s is not a palindrome.\n", string);

return 0;

}
```

OUTPUT:

input: enter a string :madam

output: the length of the string ‘madam’= 5madam is a palindrome

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 5 d	STRING MANIPULATION- PROGRAM TO COUNT THE NUMBER OF LINES, WORDS AND CHARACTERS IN A GIVEN TEXT
Date:	

AIM

To count the number of lines, words and characters in a given list.

ALGORITHM

Step 1: Start

Step 2: Read the text until an empty line

Step 3: Compare each character with newline char „\n‟ to count no offline

Step 4: Compare each character with tab char „\t\‟ or space char „ „ to count no of words

Step 5: Compare first character with NULL char „\0‟ to find the end of text

Step 6: No of characters = length of each line of text

Step 7: Print no of lines, no of words, no of chars

Step 8: Stop.

PROGRAM

```
#include <stdio.h>
#include <string.h>
int main()
{
char line[200];
int i, end = 0, characters = 0, words = 0, lines = 0;

printf("TYPE ANY TEXT.\n");
printf("PRESS ENTER TWICE TO STOP.\n");
while (1)
{
// Read a line of text
if (fgets(line, sizeof(line), stdin) == NULL)

break;
// If only ENTER is pressed, stop
if (line[0] == '\n')
break;
// Count lines
```

```
lines++;
// Count characters (excluding newline at the end)
characters += strlen(line) - 1;
// Count words
int inWord = 0;
for (i = 0; line[i] != '\0'; i++)
{
if (line[i] == ' ' || line[i] == '\t' || line[i] == '\n')
{
inWord = 0; // end of a word
}
else if (inWord == 0)
{
inWord = 1; // start of a new word
words++;
}
}
}
printf("\nNumber of lines = %d\n", lines);
printf("Number of words = %d\n", words);
printf("Number of characters = %d\n", characters);
return 0;
}
```

OUTPUT:

TYPE ANY TEXT.

PRESS ENTER TWICE TO STOP.

input : C programming is fun output: number of

Characters = 21

number of Words = 4 //(C, programming, is, fun) number of Lines = 1

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 6 a	POINTERS- TO ILLUSTRATE POINTERS TO STRUCTURES
Date:	

AIM

To write a c program to print employee details using pointers to structures.

ALGORITHM

Step 1: Start

Step 2: declare and initialize a structure with pointer

Step 3: Take input from user with scanf()

Step 4: display the output with printf()

Step 5: Stop

PROGRAM

```
#include <stdio.h>
#include <string.h>
// Creating Structure Student
struct Student
{
int roll_no;
char name[30];
char branch[40];
int batch;
};
// Variable of structure with pointer
struct Student s, *ptr;
int main()
{
ptr = &s;
// Taking inputs
printf("Enter the Roll Number of Student: ");
scanf("%d", &ptr->roll_no);
getchar(); // Consume leftover newline
printf("Enter Name of Student: ");
fgets(ptr->name, sizeof(ptr->name), stdin);
ptr->name[strcspn(ptr->name, "\n")] = '\0';  // removenewline
printf("Enter Branch of Student: ");
```

```
fgets(ptr->branch, sizeof(ptr->branch), stdin);
ptr->branch[strcspn(ptr->branch,      "\n")]    =          '\0';
printf("Enter Batch: ");
scanf("%d", &ptr->batch);
// Displaying inputs
printf("\nStudent Details:\n");
printf("Roll Number: %d\n", ptr->roll_no);
printf("Name              : %s\n", ptr->name);
printf("Branch                        : %s\n", ptr->branch);
printf("Batch  : %d\n", ptr->batch);
return 0;
}
```

OUTPUT:

input:
enter the roll number of student 1

enter name of student ABC

enter branch of student IT

enter batch of student 2025

output:

student details are:

roll no: 1

name: ABC

branch: IT

batch: 2025

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 6 b	POINTERS- POINTER ADDITION
Date:	

AIM

To write a program to perform addition using pointers.

ALGORITHM

Step 1.start

Step 2.declare and initialize the required integers and pointers

Step 3.use pointer arithmetic addition to add the values.

Step 4.use printf() to display the result

Step 5.stop

PROGRAM

```
#include <stdio.h>
int main()
{
// Integer variable
int N = 4;
// Pointer to an integer
int *ptr1, *ptr2;
// Pointer stores the address of N
ptr1 = &N;
ptr2 = &N;
printf("Pointer ptr2 before addition: %p\n", (void*)ptr2);
// Addition of 3 to ptr2
ptr2 = ptr2 + 3;  // Moves 3 integers ahead (3 * sizeof(int) bytes)

printf("Pointer ptr2 after addition: %p\n", (void*)ptr2);
return 0;
}
```

OUTPUT:

pointer ptr2 before addition: 0x7ffca373da9c

pointer ptr2 after addition: 0x7ffca373daa8

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 6 c	POINTERS- POINTER SUBTRACTION
Date:	

AIM

To write a c program to demonstrate the use of pointer subtraction.

ALGORITHM

Step 1. Start

Step 2. Declare and initialize the required integers and pointers

Step 3.use pointer arithmetic subtraction to add the values.

Step 4.Use printf() to display the result

Step 5. Stop

PROGRAM

```
#include <stdio.h>
int main()
{
int N = 4;
int *ptr1, *ptr2;
// Pointer stores the address of N
ptr1 = &N;
ptr2 = &N;
printf("Pointer ptr2 before subtraction: %p\n", (void*)ptr2);
// Subtraction of 3 from ptr2
ptr2 = ptr2 - 3;  // Moves 3 integers back (3 * sizeof(int) bytes)
printf("Pointer ptr2 after subtraction: %p\n", (void*)ptr2);
return 0;
}
```

OUTPUT:

pointer ptr2 before Subtraction: 0x7ffd718ffebc

pointer ptr2 after Subtraction: 0x7ffd718ffeb0

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 7	DYNAMIC MEMORY- TO DISPLAY ARRAY ELEMENTS USING CALLOC( ) FUNCTION.
Date:	

AIM

To write a C program to find sum of n elements entered by user. To perform this program, allocate memory dynamically using calloc() function.

ALGORITHM

Step 1: Declare structure class

Step 2: declare the variables inside the structure

Step 3: In main function declare array of structure elements

Step 4: get the input values

Step 5: using structure print the values.

PROGRAM

```
#include <stdio.h>
struct class
{
int roll_no;

char grade;

float marks;
};
// Function to display the contents of the array of structures
void display(struct class class_record[3])
{
int i;
int len = 3;
for (i = 0; i < len; i++)
{
printf("Roll number : %d\n", class_record[i].roll_no);

printf("Grade    : %c\n", class_record[i].grade);

printf("Average marks: %.2f\n", class_record[i].marks);

printf("\n");
}
```

```
}
int main()
{
// Initialize an array of structures struct class
class_record[3] = {
{ 1, 'A', 89.5f },
{ 2, 'C', 67.5f },
{ 3, 'B', 70.5f }
};
// Function call to display the class record
display(class_record);
return 0;
}
```

OUTPUT:

```
roll number :1

grade: A

average marks : 89.50

roll number :2

grade: C
average marks : 67.50

 roll number :3

grade: B
average marks : 70.50
```

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 8 a	STRUCTURES- INTERNAL MARKS OF STUDENTS
Date:	

AIM

To write a C Program to compute internal marks of students for five different subjects using structures and functions.

ALGORITHM

Step 1. Start

Step 2. Declare variables

Step 3. Read the number of students.

Step 4. Read the student mark details

Step 5. Calculate internal mark by i=total of three test marks / 3 for each subject per student.

Step 6. Display the output of the calculations for all the students.

Step 7. Stop

PROGRAM

```
#include <stdio.h>
 #include <string.h>
Struct marksheet
{
char name[20];
long int rollno;
int marks[5];
int total;
float average;
char rem[10];
char cl[20];
}
students[100];
int main()
{
int n, i, j;
char ch;
printf("How many students: ");
```

```
scanf("%d", &n);
for (i = 0; i < n; ++i)
{
printf("\nEnter details for student %d:\n", i + 1);
printf("Name: ");
scanf("%s", students[i].name);
printf("Roll Number: ");
scanf("%ld", &students[i].rollno);
students[i].total = 0;
int flag = 1; // Pass flag
for (j = 0; j < 5; ++j)
{
printf("Enter mark of subject %d: ", j + 1);
scanf("%d", &students[i].marks[j]);
students[i].total += students[i].marks[j];
if (students[i].marks[j] < 40)
{
flag = 0;
}
}
students[i].average = (float)students[i].total / 5.0;
if (flag)
{
if (students[i].average >= 75)
strcpy(students[i].cl, "Distinction");
else if (students[i].average >= 60)
strcpy(students[i].cl, "First Class");
else if (students[i].average >= 50)
strcpy(students[i].cl, "Second Class");
else if (students[i].average >= 40)
strcpy(students[i].cl, "Third Class");
strcpy(students[i].rem, "Pass");
}
```

```
else
{
strcpy(students[i].cl, "Fail");
strcpy(students[i].rem, "Fail");
}
}
for (i = 0; i < n; ++i)
{
printf("\n\n\t\t\tMark Sheet");
printf("\nName: %s\t\t Roll No: %ld", students[i].name, students[i].rollno);

 printf("\n-----------------------------------");
for (j = 0; j < 5; ++j)
{
printf("\nSubject %d : %d", j + 1, students[i].marks[j]);
}
printf("\n____________________  ");
printf("\nTotal Marks: %d\t Average: %.2f", students[i].total,  students[i].average);
printf("\nClass: %s\t Status: %s\n", students[i].cl, students[i].rem);
printf("\nPress Y to continue... ");
scanf(" %c", &ch);
if (ch != 'y' && ch != 'Y')
break;
}
return 0;
}
```

OUTPUT:

input:

How many students: 2 Enter details for student 1:

Name: Alice

Roll Number: 101
Enter mark of subject 1: 80
Enter mark of subject 2: 75
Enter mark of subject 3: 68
Enter mark of subject 4: 90
Enter mark of subject 5: 85

Enter details for student 2: Name: Bob

Roll Number: 102
Enter mark of subject 1: 45
Enter mark of subject 2: 55
Enter mark of subject 3: 60
Enter mark of subject 4: 30
Enter mark of subject 5: 50

output:

Mark Sheet

Name: Alice Roll No: 101

Subject 1 : 80

Subject 2 : 75

Subject 3 : 68

Subject 4 : 90

Subject 5 : 85

Total Marks: 398 Average: 79.60 Class:

Distinction Status: Pass

```
Press Y to continue...

Mark Sheet

Name: Bob       Roll No: 102

---------------------------------------

Subject 1 : 45

Subject 2 : 55

Subject 3 : 60

Subject 4 : 30

Subject 5 : 50

---------------------------------------

Total Marks: 240          Average: 48.00 Class: Fail

                Status: Fail
```

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 8 b	PROGRAM TO USE STRUCTURES AND UNIONS
Date:	

AIM

To write a C program that demonstrates the use of structures and unions, and to highlight the difference in memory usage and data storage between them.

ALGORITHM

Step 1: Start the program.

Step 2: Define a struct Student with members: id, name, and marks.

Step 3: Define a union Data with members: int, float, and char[].

Step 4: In the main() function:

- Declare a structure variable and assign values to its members.
- Print the structure members (all values remain intact).
- Declare a union variable and assign values one by one to its members.
- Print the union members after each assignment to observe overwriting.

Step 5: Stop the program.

PROGRAM

```
#include <stdio.h>
#include <string.h>
// Structure definition: each member has its own memory
struct Student
{
int id;
char name[20];
float marks;
};
// Union definition: all members share the same memory
union Data
{
int  i;
```

```
float  f;
char str[20];
};
int main()
{
// ===== Structure Example =====
struct Student s1;
s1.id = 101;
strcpy(s1.name, "Alice");
s1.marks = 89.5;
printf("Structure Example:\n");
printf("ID: %d\n", s1.id);
printf("Name: %s\n", s1.name);
printf("Marks: %.2f\n\n", s1.marks);
// ===== Union Example =====
union Data d1;
// Assign integer
d1.i = 10;
printf("Union Example:\n");
printf("d1.i = %d\n", d1.i);
// Assign float (overwrites previous value)
d1.f = 220.5;
printf("d1.f = %.2f\n", d1.f);
// Assign string (overwrites previous value)
strcpy(d1.str, "Hello");
printf("d1.str = %s\n", d1.str);
// Observe overwritten values
printf("\nAfter assigning string:\n");
printf("d1.i = %d (overwritten)\n", d1.i);
printf("d1.f = %.2f (overwritten)\n", d1.f);
return 0;
}
```

OUTPUT:

After assigning string:

d1.i = SomeGarbageValue <-- overwritten by string data

d1.f = SomeGarbageValue <-- overwritten by string data

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 9 a	FILES- TO DISPLAY THE CONTENT OF A FILE
Date:	

AIM

To write a c program to display the contents of a file.

ALGORITHM

Step 1: Start
Step 2: declare and initialize the required variables and file pointers
Step 3: get the file name from user input
Step 4: open the file with fopen() and read and display the contents with a while loop
Step 5: close the file
Step 6: Stop

PROGRAM

```
#include <stdio.h>
#include <string.h>
int main()
{
FILE *fs;
char ch;
char fname[100];
printf("Enter the file name: ");
fgets(fname, sizeof(fname), stdin);
fname[strcspn(fname, "\n")] = '\0';
// Remove newline
fs = fopen(fname, "r");
if (fs == NULL)
{
printf("Source file cannot be opened.\n");
return 1;
}
printf("\nContents of the file:\n");
while ((ch = fgetc(fs)) != EOF)
{
putchar(ch);
}
fclose(fs);
return 0;
}
```

OUTPUT:

input: enter the file name: sample.txt

output: this is my first program

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 9 b	FILES: TO COPY THE CONTENTS OF ONE FILE TO ANOTHER
Date:	

AIM

To write a C program to copy the contents of one file to another.

ALGORITHM

Step 1: Start

Step 2: read command line arguments

Step 3: check if no of arguments =3 or not. If not print invalid no of arguments

Step 4: open source file in read mode

Step 5: if NULL pointer, then print source file cannot be open

Step 6: open destination file in write mode

Step 7: if NULL pointer, then print destination file cannot be open

Step 8: read a character from source file and write to destination file until EOF

Step 9: Close source file and destination file

Step 10: Stop

PROGRAM

```
#include<stdio.h>
#include <stdlib.h>
 int main()
{
FILE *fs, *ft;
int c;
int count = 0;
// Open source file for reading
fs = fopen("a.txt", "r");
if (fs == NULL)
{
printf("Source file opening error\n");
return 1;
}
// Open target file for writing
ft = fopen("b.txt", "w");
if (ft == NULL)
{
printf("Target file opening error\n");
fclose(fs);
return 1;
```

```
}
// Copy file character by character
while ((c = fgetc(fs)) != EOF)
{
fputc(c, ft);
count++;
}
printf("%d bytes copied from 'a.txt' to 'b.txt'\n", count);
// Close files
fclose(fs);
fclose(ft);
return 0;
}
```

OUTPUT:

input:
a.txt

an array is a collection of elements of similar datatypes

output:

57 bytes copied from ‘a.txt’ to ‘b.txt’ 2 files closed

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 9 c	FILES: TO REVERSE THE FIRST N CHARACTERS IN A FILE.
Date:	

AIM

To write a C program to reverse the first n characters in a file.

ALGORITHM

Step 1: start

Step 2: read the command line arguments

Step 3: check if arguments=3 or not If not print invalid no of arguments

S tep 4: open source file in read mode

Step 5: if null pointer, then print file cannot be open

Step 6: store no of chars to reverse in k K= *argv[2]-48

Step 7: read the item from file stream using fread

Step 8: store chars from last position to initial position in another string(temp) step 9: print the temp string

Step 10: stop

PROGRAM

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>  // for strlen, strcspn, etc.

int main()

{

char filename[100];
char a[100];      // buffer to read characters
char s[100];      // buffer to store reversed characters

int k, i, j = 0;

size_t n;// use size_t for fread result

FILE *fp;

// Ask user for filename

printf("Enter the filename: ");

if (fgets(filename, sizeof(filename), stdin) == NULL)

{
printf("Error reading filename.\n");

return 1;
```

```
}
filename[strcspn(filename, "\n")] = '\0';
// Remove newline:contentReference[oaicite:0]{index=0}
// Ask user for number of characters to read
printf("Enter the number of characters to read: ");
if (scanf("%d", &k) != 1 || k <= 0 || k >= (int)sizeof(a))
{
printf("Invalid number of characters.\n");
return 1;
}
// Clear newline left in input buffer by scanf int ch;
while ((ch = getchar()) != '\n' && ch != EOF) { }
// Open the file for reading
fp = fopen(filename, "r");
 if (fp == NULL)
{
printf("File '%s' cannot be opened.\n", filename);
return 1;
}
```

OUTPUT:

input :

source.c

this is source

output:

command line arguments

ecruos si siht

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 9 d	FILES: TO MERGE TWO FILES INTO A THIRD FILE
Date:	

AIM

To write a C program to merge two files into a third file.

ALGORITHM

Step 1. Start
Step 2. Declare and initialize the required file pointers and variables
Step 3. Get the file name inputs of the three files from user
Step 4. Open the file1 and file2 in read mode and file3 in write mode
Step 5. Read and write the contents of file1 and file2, with a while loop to the file3
Step 6. Display the success message
Step 7. Close all the files
Step 8. Stop

PROGRAM

```
#include<stdio.h>
#include <stdlib.h>
int main()
{
FILE *fold1, *fold2, *fnew;
int ch;  // Use int to handle EOF correctly
char fname1[100], fname2[100], fname3[100];
printf("\nMerge two files and write it into a new file\n");
// Get filenames from user
printf("Input the 1st file name: ");
scanf("%s", fname1);
printf("Input the 2nd file name: ");
scanf("%s", fname2);
printf("Input the new file name where to merge the above two files: ");
scanf("%s", fname3);
// Open source files
fold1 = fopen(fname1, "r");
if (fold1 == NULL)
{
perror("Error opening first file");
exit(EXIT_FAILURE);
}
fold2 = fopen(fname2, "r");
if (fold2 == NULL)
{
```

```
perror("Error opening second file");
fclose(fold1);
exit(EXIT_FAILURE);
}
// Open target file
fnew = fopen(fname3, "w");
if (fnew == NULL)
{
perror("Error creating new file");
fclose(fold1);
fclose(fold2);
exit(EXIT_FAILURE);
}

// Copy contents of first file
while ((ch = fgetc(fold1)) != EOF)
{
fputc(ch, fnew);
}
// Copy contents of second file
while ((ch = fgetc(fold2)) != EOF)
{
fputc(ch, fnew);
}
printf("The two files have been merged into '%s' successfully!\n", fname3);
// Close all files
fclose(fold1);
fclose(fold2);
fclose(fnew);
return 0;
}
```

OUTPUT:

Input the 1st file name : test.txt

Input the 2nd file name : test1.txt

Input the new file name where to merge the above two files :

mergefiles.txt

The two files merged into mergefiles.txt file successfully..!!

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 10 a	USE OF STANDARD AND USER-DEFINED LIBRARIES IN SOLVING PROBLEMS-SQUARE ROOT (STANDARD) & FACTORIAL (USER DEFINED)
Date:	

AIM

To demonstrate the use of standard library math.h for square root calculation and a user-defined library for factorial calculation.

ALGORITHM

Step 1: Start the program.
Step 2: Include <stdio.h>, <math.h>, and "mylib1.h".
Step 3: Use sqrt() from <math.h> to calculate square root.
Step 4: Use factorial() from user-defined library to calculate factorial.
Step 5: Display the results.
Step 6: End program.

PROGRAM

```
#include <stdio.h>
#include <math.h>
// Factorial function int
factorial(int n)
{
if (n <= 1)
return 1;
else
return n * factorial(n - 1);
}
int main()
{
int num = 5;
double x = 25.0;
// Using math.h function
printf("Square root of %.2f = %.2f\n", x, sqrt(x));
// Using custom factorial function
printf("Factorial of %d = %d\n", num, factorial(num));
return 0;
}
```

OUTPUT:

Square root of 25.00 = 5.00

Factorial of 5 = 1

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 10 b	USE OF STANDARD AND USER-DEFINED LIBRARIES IN SOLVING PROBLEMS- POWER FUNCTION (STANDARD)
Date:	

AIM

To demonstrate the use of standard library math.h for power calculation and a user-defined library for finding GCD of two numbers.

ALGORITHM

Step 1: Start the program.
Step 2: Include <stdio.h>, <math.h>, and "mylib2.h".
Step 3: Use pow() from <math.h> to calculate power.
Step 4: Use gcd()from user-defined library.
Step 5: Display results.
Step 6: End program.

PROGRAM

```
#include <stdio.h>

#include <math.h>

// GCD function using Euclidean algorithm

int gcd(int a, int b)

{

if (b == 0)

return a;

 else

return gcd(b, a % b);

}

int main()

{

int a = 48, b = 18;

double base = 2.0, exp = 5.0;

// Power calculation

printf("%.2f raised to power %.2f = %.2f\n", base, exp, pow(base, exp));

// GCD calculation

printf("GCD of %d and %d = %d\n", a, b, gcd(a, b));

return 0;

}
```

OUTPUT:

2 raised to power 5 = 32.00

GCD of 48 and 18 = 6

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 10 c	USE OF STANDARD AND USER-DEFINED LIBRARIES IN SOLVING PROBLEMS- STRING LENGTH (STANDARD)
Date:	

AIM

To demonstrate the use of standard library string.h for string length calculation and a user-defined library to check if a string is a palindrome.

ALGORITHM

Step 1: Start the program.
Step 2: Include <stdio.h>, <string.h>, and "mylib3.h".
Step 3: Use strlen() from <string.h> to find length.
Step 4: Use isPalindrome() from user-defined library.
Step 5: Display results.
STEP 6: End program.

PROGRAM

```
#include <stdio.h>
 #include <string.h>
#include <stdbool.h>
// Function to check palindrome bool
isPalindrome(const char *str)
{
int left = 0;
int right = strlen(str) - 1;
while (left < right)
{
if (str[left] != str[right])
 return false;
left++;
right--;
}
return true;
}
int main()
{
char word[] = "madam";
```

```
printf("Length of string '%s' = %lu\n", word, strlen(word));
if (isPalindrome(word))
printf("'%s' is a palindrome\n", word);
 else
printf("'%s' is not a palindrome\n", word);
return 0;
}
```

OUTPUT:

Length of string 'madam' = 5

'madam' is a pal

RESULT:

The C program was executed successfully, and its output was verified.

Ex.No: 11 a	MINI PROJECT CREATE A —RAILWAY RESERVATION SYSTEM.
Date:	

AIM

Create a Railway reservation system in C with the following modules

- Booking
- Availability checking
- Cancellation
- Prepare chart

ALGORITHM

1.Start

2.Declare variables

3.Display the menu options

4.Read the option.

5.Develop the code for each option.

6.Display the output of the selected option based on existence.

7.Stop

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
#include <string.h>
int first = 5, second = 5, third = 5;
struct node
{
int ticketno;
long phoneno;
char name[100];
char address[100];
}s[15];
int i = 0;
void booking()
{
if (i >= 15)
{
printf("Booking full!\n");
```

```
return;
}
printf("\nEnter your details:\n");
printf("Name: ");
scanf("%s", s[i].name);
printf("Phone number: ");
scanf("%ld", &s[i].phoneno);
printf("Address: ");
scanf("%s", s[i].address);
printf("Ticket number (1-10): ");
scanf("%d", &s[i].ticketno);
i++;
printf("Booking successful!\n");
}
void availability()
{
int c;
printf("\nAvailability Checking:\n");
printf("1. First class\n2. Second class\n3. Third class\n");
printf("Enter the option: ");
scanf("%d", &c);
switch (c)
{
case 1:
if (first > 0)
{
printf("Seat available\n");
first--;
}
else
{
printf("Seat not available\n");
}
break;
case 2:
```

```
if (second > 0)
{
printf("Seat available\n");
second--;
}
else
{
printf("Seat not available\n");
}
break;

case 3:
if (third > 0)
{
printf("Seat available\n");
third--;
}
else
{
printf("Seat not available\n");
}
break;

 default:
printf("Invalid option!\n");
break;
}
}
void cancel()
{
int c;
printf("\nCancel Ticket\n");
printf("Which class do you want to cancel?\n1. First class\n2. Second class\n3. Third
class\n");
printf("Enter the option: ");
scanf("%d", &c);
switch (c)
```

```
{
case 1:
first++;
break;
case 2:
second++;
 break;
case 3:
third++;
break;
default:
printf("Invalid option!\n");
return;
}
printf("Ticket is canceled\n");
}
void chart()
{
int c;
printf("\n----- Ticket Chart  \n");
printf("Ticket No\tName\n");

for (c = 0; c < i; c++)
{
printf("%d\t\t%s\n", s[c].ticketno, s[c].name);
}
}
int main()
{
int n;
printf("Welcome to Railway Ticket Reservation System\n");
while (1)
{
printf("\n1. Booking\n2. Availability Checking\n3. Cancel\n4. Chart\n5. Exit\nEnter your option: ");
scanf("%d", &n); switch (n)
{
case 1:
```

```
booking();

break;

case 2:

availability();

break;

case 3:
cancel();

break;

case 4:

chart();

break;

case 5:

printf("Thank you, visit again!\n");

exit(0);

default:

printf("Invalid option!\n");

break;

}

}
return 0;

}
```

OUTPUT:

Welcome to Railway Ticket Reservation System

1. Booking
2. Availability Checking
3. Cancel
4. Chart
5. Exit

Enter your option: 1

Enter your details:

Name: Alice

Phone number: 9876543210

Address: Chennai

Ticket number (1-10): 101

Booking successful!

1. Booking
2. Availability Checking
3. Cancel
4. Chart
5. Exit

Enter your option: 2

Availability Checking:

1. First class
2. Second class
3. Third class

Enter the option: 1

Seat available

1. Booking
2. Availability Checking
3. Cancel

4. Chart

5. Exit

Enter your option: 4

----- Ticket Chart ----- Ticket

No Name

101 Alice

1. Booking

2. Availability Checking

3. Cancel

4. Chart

5. Exit

Enter your option: 3

Cancel Ticket

Which class do you want to cancel?

1. First class

2. Second class

3. Third class

Enter the option: 1

Ticket is canceled

1. Booking

2. Availability Checking

3. Cancel

4. Chart

5. Exit

Enter your option: 5

Thank you, visit again!s

RESULT

The C program was executed successfully, and its output was verified.

Ex.No: 11 b	BANKING APPLICATION
Date:	

AIM

To write a C Program to Count the number of account holders whose balance is less than the minimum balance using sequential access file.

ALGORITHM

1.Start

2.Declare variables and file pointer.

3.Display the menu options.

4.Read the Input for transaction processing.

5.Check the validation for the input data.

6.Display the output of the calculations.

7.Repeat step 3 until choose to stop.

8.Stop

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct bank
{
  int no;
  char name[20];
  float dep;
} s[100];
int a = 0;
int nextAcc = 1001;
void creation()
{
printf("\n********** NEW ACCOUNT CREATION **********\n");
printf("Your account number is: %d\n", nextAcc);
s[a].no = nextAcc;
printf("Enter your name: ");
scanf("%19s", s[a].name);          // limit input to avoid overflow
printf("Enter initial deposit (minimum Rs.500): ");
```

```
scanf("%f", &s[a].dep);
 if (s[a].dep < 500.0f)
{
printf("Deposit less than minimum. Setting balance to Rs.500.\n");
s[a].dep = 500.0f;
}
a++;
nextAcc++;
}
void deposit()
{
int no, idx = -1;
float  amount;
printf("\n********** CASH DEPOSIT **********\n");
printf("Enter your account number: ");
scanf("%d", &no);
for (int i = 0; i < a; i++)
{
if (s[i].no == no)
{
idx = i;
break;
}
}
if (idx != -1)
{
printf("Account No: %d\nName: %s\nBalance: %.2f\n", s[idx].no, s[idx].name, s[idx].dep);
printf("Enter deposit amount: ");
scanf("%f", &amount);
if (amount < 0)
{
printf("Invalid amount. Deposit cancelled.\n");
}
```

```
else
{
s[idx].dep += amount;
printf("New balance: %.2f\n", s[idx].dep);
}
}
else
{
printf("Account number invalid.\n");
}
}
void withdraw()
{
int no, idx = -1;
float amount;
printf("\n********** CASH WITHDRAW **********\n");
printf("Enter your account number: ");
scanf("%d", &no);
for (int i = 0; i < a; i++)
{
if (s[i].no == no)
{
idx = i;
break;
}
}
if (idx != -1)
{
printf("Account No: %d\nName: %s\nBalance: %.2f\n", s[idx].no, s[idx].name, s[idx].dep);
printf("Enter withdraw amount: ");
scanf("%f", &amount);
if (amount < 0)
{
```

```
printf("Invalid amount. Withdrawal cancelled.\n");
}
else if (s[idx].dep - amount < 500.0f)
{
printf("Cannot withdraw. Minimum balance Rs.500 required.\n");
}
else
{
s[idx].dep -= amount;
printf("New balance: %.2f\n", s[idx].dep);
}
}
else
{
printf("Account number invalid.\n");
}
}
void lowbal()
{
printf("\n********** ACCOUNTS WITH BALANCE < 1000 **********\n");
for (int i = 0; i < a; i++)
{
if (s[i].dep < 1000.0f)
{
printf("Account No: %d\tName: %s\tBalance: %.2f\n", s[i].no, s[i].name, s[i].dep);
}
}
}
int main()
{
int ch;
```

```
do
{
printf("\n********** BANKING SYSTEM **********\n");
printf("1. Create new account\n");
printf("2. Cash deposit\n");
printf("3. Cash withdraw\n");
printf("4. Low balance enquiry\n");
printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &ch);
switch (ch)
{
 case 1:
 creation();
  break;
 case 2:
 deposit();
 break;
 case 3:
 withdraw();
 break;
 case 4:
 lowbal();
 break;
 case 5:
 printf("Thank you for using the banking system!\n");
 break;
 default:
 printf("Invalid choice. Try again.\n");
 break;
 }
 } while (ch != 5);
 return 0;
 }
```

OUTPUT:

********** BANKING SYSTEM **********

1. Create new account
2. Cash deposit
3. Cash withdraw
4. Low balance enquiry
5. Exit

Enter your choice: 1

********** NEW ACCOUNT CREATION ********** Your account number is: 1001

Enter your name: Alice

Enter initial deposit (minimum Rs.500): 600

********** BANKING SYSTEM **********

1. Create new account
2. Cash deposit
3. Cash withdraw
4. Low balance enquiry
5. Exit

Enter your choice: 1

********** NEW ACCOUNT CREATION ********** Your account number is: 1002

Enter your name: Bob

Enter initial deposit (minimum Rs.500): 400

Deposit less than minimum. Setting balance to Rs.500.

********** BANKING SYSTEM **********

```
Enter your choice: 2

********** CASH DEPOSIT **********
Enter your account number: 1001 Account No: 1001
Name: Alice Balance: 600.00
Enter deposit amount: 200
New balance: 800.00

********** BANKING SYSTEM **********
Enter your choice: 3

********** CASH WITHDRAW **********
Enter your account number: 1001
Account No: 1001
Name: Alice
Balance: 800.00
Enter withdraw amount: 400
Cannot withdraw. Minimum balance Rs.500 required.

********** BANKING SYSTEM **********
Enter your choice: 4

********** ACCOUNTS WITH BALANCE < 1000 **********
Account No: 1001    Name: Alice        Balance: 800.00
Account No: 1002    Name: Bob          Balance: 500.00


********** BANKING SYSTEM **********
Enter your choice: 5
Thank you for using the banking system!
```

RESULT

The C program was executed successfully, and its output was verified.